

Knowing When to Act: A Hybrid Classification–Generation System for Proactive Audio Engagement

Bailey Malota
Computer Science
Virginia Tech
Springfield, Virginia
baileym04@vt.edu

Jaemyung Yu
Computer Science
Virginia Tech
Arlington, Virginia
jaemyungy25@vt.edu

Martin Balla
Computer Science
Virginia Tech
Arlington, Virginia
martinb04@vt.edu

Maximilian Ilyev
Computer Science
Virginia Tech
Sterling, Virginia
maximilianilyev@vt.edu

Abstract—Audio-aware assistants in homes, workplaces, and care settings must decide not only what they hear but whether responding would help. We propose a system for Track II (Proactive Audio Engagement) of the HEARTBEAT Challenge [1], in which a system hears 10–90 s of environmental or human non-speech audio and must choose among silence, monitor, ask, and intervene, decide when to engage, and produce a concrete response with a brief rationale. Our approach decouples deciding from speaking: a frozen pretrained audio encoder feeds a lightweight causal temporal model that predicts actions frame by frame and derives the engagement time, while a compact language model fine-tuned with LoRA generates the response conditioned on the predicted action and detected sound events. A template-based generator serves as an internal baseline, allowing a controlled comparison of retrieval versus generation. We will evaluate on the challenge development data and leaderboard, with particular attention to false activations and robustness under distribution shift.

Index Terms—Proactive audio engagement, Sound event detection, Audio-language models, Large language models, Abstention

I. INTRODUCTION

Smart speakers, home-monitoring devices, and assistive technologies increasingly listen continuously to their surroundings. Their value depends on noticing the moments that matter—a smoke alarm, a kettle left boiling, a fall, a person coughing or crying—while staying quiet the rest of the time. Both failure modes are costly. A missed alarm can be dangerous, while an assistant that interrupts over every door slam or a sound that has already stopped quickly becomes an annoyance that users ignore or switch off. Modern audio models are good at recognizing what a sound is, but recognition alone does not tell a system whether, when, or how to act.

The HEARTBEAT Challenge [1] formalizes this gap. In Track II, the input is a 10–90 s audio segment of environmental or human non-speech sound sampled at 16 kHz. For each segment, a system must (i) choose an appropriate engagement time, (ii) predict one of four actions: silence, monitor, ask, or intervene, and (iii) when acting, produce a concrete response and a brief rationale. Positive cases include safety alarms, door cues, appliance states, impacts, and human non-speech cues. Hard negatives include weak evidence, premature moments, already-resolved events, and scenes where no action is

warranted. Appropriate abstention is explicitly rewarded, and false-activation rate serves as a tie-breaker.

This problem is well suited to deep learning. It requires learned acoustic representations robust to varied devices and environments, temporal modeling of how a scene unfolds, and natural-language generation grounded in audio evidence. Our goals for the semester are specific and staged:

- 1) Reproduce the organizer baseline and build an internal evaluation pipeline on the development set (Checkpoint 1).
- 2) Deliver a complete end-to-end hybrid system with template-based responses (Checkpoint 2).
- 3) Replace templates with a fine-tuned language-model generator and quantify the gain.
- 4) Submit a valid final container to the challenge and analyze performance under distribution shift.

II. RELATED WORK

A. Audio Representation and Event Detection

Large-scale weakly labeled data such as AudioSet [2] enabled general-purpose audio encoders, including CNN-based PANNs [3], the Audio Spectrogram Transformer (AST) [4], and self-supervised BEATs [5]. CLAP [6] aligns audio with text in a shared embedding space. Sound event detection, advanced largely through the DCASE challenges [7], localizes events in time. These models answer what happened and when, which is necessary but not sufficient for our task. We use them as frozen feature extractors.

B. Audio-Language Models

Audio captioning datasets such as AudioCaps [8] and Clotho [9] link audio to natural language. More recent audio-language models connect an audio encoder to a large language model (LLM): Pengi [10], LTU [11] (built on LLaMA), SALMONN [12] (built on Vicuna), and Qwen2-Audio [13] (built on Qwen). These models can describe and reason about audio, but they are trained to answer when asked and not to decide whether an unprompted response is warranted. They are also large for a no-network, container-based evaluation.

C. Turn-Taking and Proactive Agents

Deciding when to speak has been studied mainly for conversation, from turn-taking models [14] to self-supervised voice activity projection [15]. Proactive dialogue systems [16] and proactive LLM agents [17] study when an assistant should take initiative, but they operate on text or on user activity, not on environmental audio.

D. Our Contribution

Prior work either detects and describes sounds or decides when to speak in a conversation. Track II combines both problems in non-speech scenes. Our contribution is (i) a hybrid architecture that separates the engagement decision from response generation and conditions generation on that decision to keep actions and responses consistent, (ii) a controlled comparison between template retrieval and LLM generation on the same decision backbone, and (iii) an analysis of false activations and robustness on the challenge’s generalization split.

III. PROPOSED APPROACH

Our system has three stages (Fig. 1). The first two produce the timing and action outputs; the third produces the response and rationale.

A. Stage 1: Frozen Audio Encoder

Each clip is split into overlapping windows (for example, a 1 second window with a 0.5 second hop), and each window is embedded by a pretrained AudioSet encoder. Candidates include BEATs [5], AST [4], PANNs [3], and CLAP [6]. The choice will be made empirically on development data. Freezing the encoder lets us compute embeddings, together with the encoder’s AudioSet event probabilities, once and cache them. All later experiments then train on small cached tensors rather than raw audio, which makes iteration fast and conserves shared GPU resources. Fine-tuning the encoder is a stretch goal.

B. Stage 2: Causal Decision Model

A small temporal model (a unidirectional GRU or a causally masked transformer) reads the embedding sequence and outputs, at every frame t , a distribution over the four actions. The engagement time $\hat{t}$ is the first frame at which a non-silence action exceeds a calibrated confidence threshold, and the predicted action is the one at $\hat{t}$. If no frame crosses the threshold, the output is silence.

Since the challenge labels each clip with a single action and engagement time, we derive per-frame training targets by labeling frames before the engagement time as silence and frames from it onward as the clip’s action, excluding a short band just before the engagement time from the loss to absorb annotation timing noise.

We make this model causal on purpose: its decision at time t sees only audio up to t . This matches the “unfolding scene” framing of the task. It also stops the model from peeking at how an event later resolves, and it remains valid whether the final evaluation is offline or streaming, which

has not yet been specified. Framing timing as “when is the evidence sufficient?” addresses the premature moment and weak evidence hard negatives directly.

C. Stage 3: Response and Rationale Generation

A short rule-based step builds a structured summary for the generator. The action and engagement time come from Stage 2, while the event list comes from Stage 1’s AudioSet class probabilities, thresholded and merged across consecutive frames into timed segments. For example: “Action: intervene. Events: smoke alarm (12–40 s, $p = 0.93$); footsteps (35 s).” Because the action is given rather than chosen by the generator, the response cannot contradict it.

Template baseline. We will cluster training-set reference responses by action and event type and use a representative response and rationale from each cluster, so templates reflect the organizers’ style.

LLM generator. A compact open LLM of roughly 1–4 B parameters will be fine-tuned with LoRA [18] on (summary, reference response, rationale) pairs. Candidates include Qwen2.5 [19], Llama 3.2 [20], and Phi-3 [21], whose families underlie several of the audio-language models above. The model will be chosen for size, license terms, and development-set quality, and declared before the challenge’s November 20 resource freeze.

D. Class Imbalance

We expect most frames, and likely many clips, to warrant no action. This imbalance affects the system in three places. First, in decision training, plain cross-entropy would bias the model toward silence. Candidate remedies are class-weighted loss, focal loss [22], and balanced clip sampling. Second, the engagement threshold trades premature or false activations against late or missed ones. We will tune it on development data, with temperature scaling [23] if the probabilities are poorly calibrated. Because false-activation rate is the tie-breaker, we will not simply maximize recall. Third, for Stage 3, until the official rules clarify otherwise, we will treat monitor as a non-response action. Text will be generated only for ask and intervene. These response-generating cases may be dominated by common events, so fine-tuning data may need rebalancing by event type. We will choose among these remedies after examining the released label distribution.

IV. SYSTEM DESIGN

A. Architecture

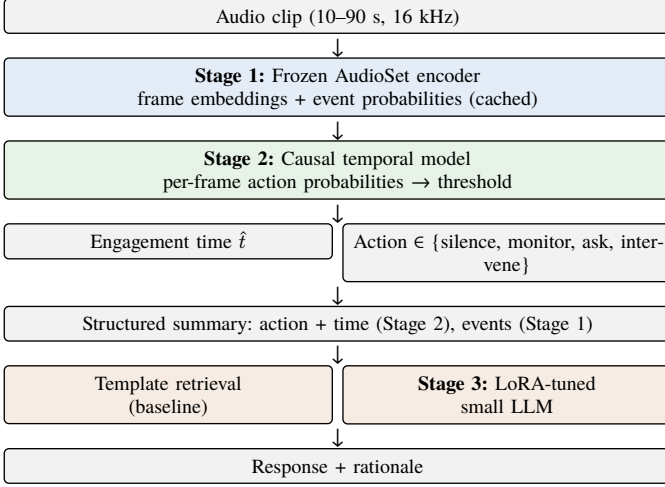


Fig. 1. Proposed hybrid pipeline. Stages 1–2 produce timing and action; Stage 3 generates the response conditioned on them.

B. Data Sets

The challenge provides labeled training and development data plus an input-only leaderboard set (Table I), all released October 14, 2026 [1]. Final ranking uses a private benchmark split into a hidden main portion (70%) that matches the training distribution and a hidden generalization portion (30%) with shifted speakers, environments, devices, sound sources, and acoustic conditions. Declared public data and pretrained models are allowed, so AudioSet-pretrained encoders and open LLMs are permitted.

TABLE I
TRACK II DATA (10–90 s SEGMENTS, 16 kHz)

Partition	Hours	Labels
Training	300	Labeled
Development	15	Labeled
Leaderboard	15	Input only

C. Baselines and Evaluation

We will compare against (i) the organizer baseline, (ii) a trivial “always silence” system as a sanity check, and (iii) our template system, which isolates the effect of LLM generation. The official Track II score is 0.30 engagement timing + 0.30 action accuracy + 0.40 response and rationale quality [1].

With only two leaderboard submissions per day, we need internal metrics on the development set. These are action top-1 accuracy with macro-F1 and a confusion matrix, which reveal imbalance effects that accuracy hides; timing accuracy across a range of tolerances; false-activation rate; and, for text, BERTScore [24] against references plus a small manual review, optionally with a local LLM-as-judge [25] as a proxy. The timing tolerance and response scoring procedure have not yet been published, so we treat these metrics as provisional. We will avoid hard design decisions that depend on them, such

as tuning heavily to one text metric, until the official scorers are released. Our internal scoring harness will parameterize timing tolerances and text-scoring choices so that the official definitions can be substituted without changing the evaluation pipeline.

Because the generalization portion of the benchmark is hidden, we will approximate it by holding out a shifted copy of the development set, created with gain changes, band-limiting, added noise, and reverberation, and will report every internal metric on both the clean and shifted versions. The development set is also small relative to the number of design choices we will compare, so we will report bootstrap confidence intervals rather than single numbers.

D. Languages, Tools, and Compute

The system will be implemented in Python with PyTorch, torchaudio, Hugging Face Transformers, and PEFT (for LoRA), with code shared on GitHub. Training runs through Slurm on Virginia Tech ARC under the course allocation. Falcon A30 GPUs will handle embedding extraction and decision-model training, while TinkerCliffs A100 GPUs will handle LLM fine-tuning; Owl B200 GPUs may be used for larger models or experiments when available. One shared copy of the data and model caches will live in group project storage, not in home directories, and GPU jobs will be submitted as batch jobs sized to what they use. Final systems must be packaged as no-network OCI containers [1], so model size and inference latency will be kept modest.

REFERENCES

- [1] HEARTBEAT Challenge Organizers, “HEARTBEAT Challenge 2027: Human-Centered Audio Agents.” [Online]. Available: <https://heartbeat-challenge.github.io/>
- [2] J. F. Gemmeke *et al.*, “Audio Set: An ontology and human-labeled dataset for audio events,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 776–780.
- [3] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, “PANNs: Large-scale pretrained audio neural networks for audio pattern recognition,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.
- [4] Y. Gong, Y.-A. Chung, and J. Glass, “AST: Audio Spectrogram Transformer,” in *Proc. Interspeech*, 2021, pp. 571–575.
- [5] S. Chen *et al.*, “BEATs: Audio pre-training with acoustic tokenizers,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.
- [6] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, “CLAP: Learning audio concepts from natural language supervision,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2023.
- [7] A. Mesaros, T. Heittola, T. Virtanen, and M. D. Plumbley, “Sound event detection: A tutorial,” *IEEE Signal Processing Magazine*, vol. 38, no. 5, pp. 67–83, 2021.
- [8] C. D. Kim, B. Kim, H. Lee, and G. Kim, “AudioCaps: Generating captions for audios in the wild,” in *Proc. NAACL-HLT*, 2019.
- [9] K. Drossos, S. Lipping, and T. Virtanen, “Clotho: An audio captioning dataset,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2020.
- [10] S. Deshmukh, B. Elizalde, R. Singh, and H. Wang, “Pengi: An audio language model for audio tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [11] Y. Gong, H. Luo, A. H. Liu, L. Karlinsky, and J. Glass, “Listen, think, and understand,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

- [12] C. Tang *et al.*, “SALMONN: Towards generic hearing abilities for large language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
- [13] Y. Chu, J. Xu, Q. Yang, and others, “Qwen2-Audio technical report,” *arXiv preprint arXiv:2407.10759*, 2024.
- [14] G. Skantze, “Turn-taking in conversational systems and human-robot interaction: A review,” *Computer Speech and Language*, vol. 67, p. 101178, 2021.
- [15] E. Ekstedt and G. Skantze, “Voice activity projection: Self-supervised learning of turn-taking events,” in *Proc. Interspeech*, 2022.
- [16] Y. Deng, W. Lei, W. Lam, and T.-S. Chua, “A survey on proactive dialogue systems: Problems, methods, and prospects,” in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2023.
- [17] Y. Lu and others, “Proactive agent: Shifting LLM agents from reactive responses to active assistance,” *arXiv preprint arXiv:2410.12361*, 2024.
- [18] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
- [19] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [20] A. Grattafiori and others, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [21] M. Abdin and others, “Phi-3 technical report: A highly capable language model locally on your phone,” *arXiv preprint arXiv:2404.14219*, 2024.
- [22] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [23] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2017.
- [24] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.
- [25] L. Zheng and others, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

APPENDIX

Task Assignment

Member	Responsibilities
Maximilian Ilyev (Lead)	Primary, evaluation and delivery: scoring harness, end-to-end parameter sweeps, offline container, challenge submissions, coordination, report editing. Secondary, Stage 2: threshold and decoder tuning against the end-to-end score.
Jaemyung Yu	Primary, Stage 1: data transfer to ARC, preprocessing, encoder selection, chunking of long clips, embedding and event-probability caching. Secondary, Stage 3: event thresholding and merging into timed segments.
Bailey Malota	Primary, Stage 2: causal temporal model, conversion of clip-level labels to per-frame targets, trigger decoding, class-imbalance handling, calibration. Secondary, Stage 1: encoder comparison and embedding extraction.
Martin Balla	Primary, Stage 3: template construction, LLM selection, LoRA fine-tuning, robustness to incorrect upstream predictions. Secondary, evaluation: text-qual-

Member	Responsibilities
	ity metrics and packaging the LLM for offline inference.
Schedule	
Dates	Milestones
Sep 22	Proposal due.
Sep 22 – Oct 13	ARC environment and shared storage; pipeline skeleton tested on public audio; offline container smoke test with representative checkpoints (image size, latency, no network access); literature review; challenge registration (opens Oct 7).
Oct 14 – 20	Data, rules, scorers, and baseline released. Exploratory analysis (label distribution, response format), baseline reproduction, embedding extraction. Checkpoint 1 (Oct 20).
Oct 21 – Nov 10	Leaderboard opens Oct 21. Train decision model; build template generator; first submissions; begin LLM selection and LoRA tuning. Checkpoint 2 (Nov 10): end-to-end system with templates.
Nov 11 – 20	Finalize LLM and pretrained-model choices, checked against measured container size and latency; team and resource declarations frozen Nov 20.
Nov 21 – Dec 4	Integrate LLM generator into the container; ablations (templates vs. LLM, encoder choice); threshold tuning; stretch: encoder fine-tuning. Final container frozen Dec 3 and validated on the leaderboard before it closes Dec 4.
Dec 5 – 11	No system changes; the leaderboard-validated container is submitted unchanged. Final report. Container and report both due Dec 11.